ANSWER SHEET

QUESTION 1 [25 MARKS]

a. Write a Python function **calculate_fee(credit_hours, fee_per_credit)** that calculates the total course fee. The function should **return the total fee**.

fee_module.py > ...

```python
def calculate_fee(credit_hours, fee_per_credit):
    return credit_hours * fee_per_credit
```

[4 marks]

b. Using a dictionary, store course fee and credit hours for each course choice (1, 2, 3).

```python

courses = {
    1: {
        "course_name": "Information Technology",
        "credit_hours": 10,
        "fee_per_credit": 150
    },
    2: {
        "course_name": "Software Engineering",
        "credit_hours": 12,
        "fee_per_credit": 180
    },
    3: {
        "course_name": "Data Science",
        "credit_hours": 15,
        "fee_per_credit": 200
    }
}

```

[4 marks]

c. Define a class **Student** with attributes such as **student_name, course_name, and credit_hours.**

```
25
26  class Student:
27      def __init__(self, student_name, course_choice, course_name,
28                   credit_hours, total_fee):
29          self.student_name = student_name
30          self.course_choice = course_choice
31          self.course_name = course_name
32          self.credit_hours = credit_hours
33          self.total_fee = total_fee
```

[5 marks]

d. Apply the following method in the **Student** class:

- **display_info()** to display student details and total fee

```
35
36      def display_info(self):
37          if self.total_fee > 2000:
38              category = "High Fee"
39          else:
40              category = "Normal Fee"
41
42          print("\nStudent Name:", self.student_name)
43          print("Course Choice:", self.course_choice)
44          print("Course Name:", self.course_name)
45          print("Credit Hours:", self.credit_hours)
46          print(f"Total Fee: RM {self.total_fee:.2f}")
47          print("Fee Category:", category)
```

[4 marks]

e. Write a program that accepts user input, calls the function, creates object, and stores multiple student records in a list by using a module file (fee_module.py).

fee_module.py | question1.py | question2.py

question1.py > ...

```python
import fee_module

students = []

while True:
    print("\nStudent Course Fee Management System")
    print("1 - Information Technology")
    print("2 - Software Engineering")
    print("3 - Data Science")

    student_name = input("\nEnter student name: ").strip()

    if student_name == "":
        print("Student name cannot be empty.")
        continue

    try:
        course_choice = int(input("Enter course choice (1-3): "))
    except ValueError:
        print("Please enter a number.")
        continue

    if course_choice not in fee_module.courses:
        print("Invalid course choice.")
        continue

    course = fee_module.courses[course_choice]

    total_fee = fee_module.calculate_fee(
        course["credit_hours"],
        course["fee_per_credit"]
    )

    student = fee_module.Student(
        student_name,
        course_choice,
        course["course_name"],
        course["credit_hours"],
        total_fee
    )

    students.append(student)
    student.display_info()

    again = input("\nAdd another student? (y/n): ").strip().lower()

    if again != "y":
        break

print("\n========All Student Records========")

for student in students:
    student.display_info()
```

[4 marks]

f. Show the **complete code** for Student Course Fee Management System

fee_module.py

```python
def calculate_fee(credit_hours, fee_per_credit):
    return credit_hours * fee_per_credit

courses = {
    1: {
        "course_name": "Information Technology",
        "credit_hours": 10,
        "fee_per_credit": 150
    },
    2: {
        "course_name": "Software Engineering",
        "credit_hours": 12,
        "fee_per_credit": 180
    },
    3: {
        "course_name": "Data Science",
        "credit_hours": 15,
        "fee_per_credit": 200
    }
}

class Student:
    def __init__(self, student_name, course_choice, course_name,
                 credit_hours, total_fee):
        self.student_name = student_name
        self.course_choice = course_choice
        self.course_name = course_name
        self.credit_hours = credit_hours
        self.total_fee = total_fee

    def display_info(self):
        if self.total_fee > 2000:
            category = "High Fee"
        else:
            category = "Normal Fee"

        print("\nStudent Name:", self.student_name)
        print("Course Choice:", self.course_choice)
        print("Course Name:", self.course_name)
        print("Credit Hours:", self.credit_hours)
        print(f"Total Fee: RM {self.total_fee:.2f}")
        print("Fee Category:", category)
```

question1.py

```python
import fee_module

students = []

while True:
    print("\nStudent Course Fee Management System")
    print("1 - Information Technology")
    print("2 - Software Engineering")
    print("3 - Data Science")

    student_name = input("\nEnter student name: ").strip()

    if student_name == "":
        print("Student name cannot be empty.")
        continue

    try:
        course_choice = int(input("Enter course choice (1-3): "))
    except ValueError:
        print("Please enter a number.")
        continue

    if course_choice not in fee_module.courses:
        print("Invalid course choice.")
        continue

    course = fee_module.courses[course_choice]

    total_fee = fee_module.calculate_fee(
        course["credit_hours"],
        course["fee_per_credit"]
    )

    student = fee_module.Student(
        student_name,
        course_choice,
        course["course_name"],
        course["credit_hours"],
        total_fee
    )

    students.append(student)
    student.display_info()

    again = input("\nAdd another student? (y/n): ").strip().lower()

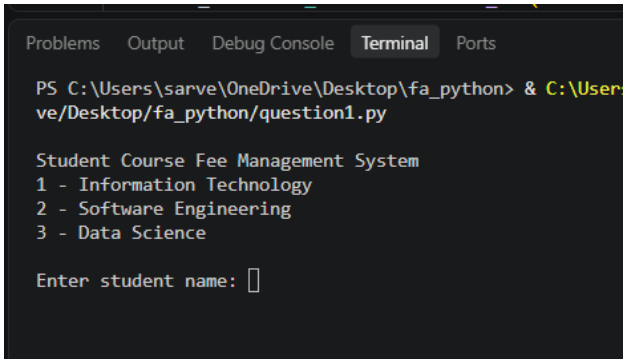
    if again != "y":
        break

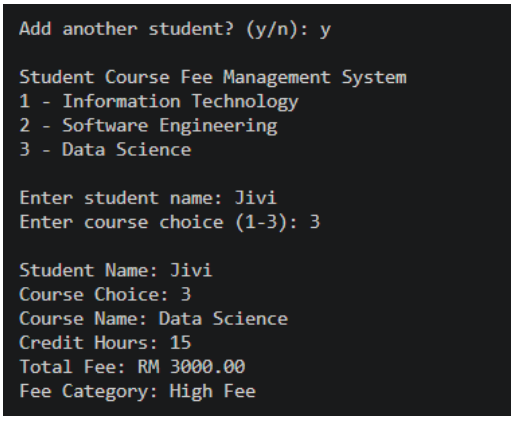
print("\n========All Student Records========")

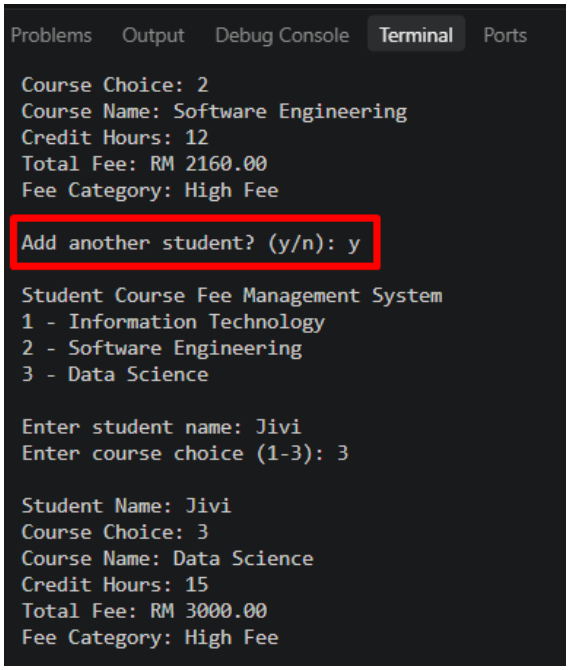
for student in students:
    student.display_info()
```

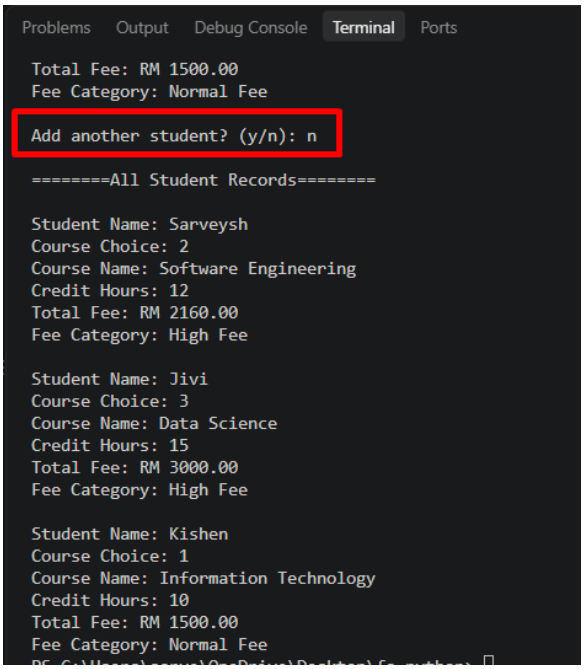
[4 marks]

Output :


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\sarve\OneDrive\Desktop\fa_python> & C:\User
ve/Desktop/fa_python/question1.py
Student Course Fee Management System
1 - Information Technology
2 - Software Engineering
3 - Data Science
Enter student name:


Add another student? (y/n): y
Student Course Fee Management System
1 - Information Technology
2 - Software Engineering
3 - Data Science
Enter student name: Jivi
Enter course choice (1-3): 3
Student Name: Jivi
Course Choice: 3
Course Name: Data Science
Credit Hours: 15
Total Fee: RM 3000.00
Fee Category: High Fee


Problems
Output
Debug Console
Terminal
Ports
Course Choice: 2
Course Name: Software Engineering
Credit Hours: 12
Total Fee: RM 2160.00
Fee Category: High Fee
Add another student? (y/n): y
Student Course Fee Management System
1 - Information Technology
2 - Software Engineering
3 - Data Science
Enter student name: Jivi
Enter course choice (1-3): 3
Student Name: Jivi
Course Choice: 3
Course Name: Data Science
Credit Hours: 15
Total Fee: RM 3000.00
Fee Category: High Fee


Problems
Output
Debug Console
Terminal
Ports
Total Fee: RM 1500.00
Fee Category: Normal Fee
Add another student? (y/n): n
========All Student Records========
Student Name: Sarveysh
Course Choice: 2
Course Name: Software Engineering
Credit Hours: 12
Total Fee: RM 2160.00
Fee Category: High Fee
Student Name: Jivi
Course Choice: 3
Course Name: Data Science
Credit Hours: 15
Total Fee: RM 3000.00
Fee Category: High Fee
Student Name: Kishen
Course Choice: 1
Course Name: Information Technology
Credit Hours: 10
Total Fee: RM 1500.00
Fee Category: Normal Fee

Rubric for Student Course Fee Management System (Total: 25 Marks)

Component	5	4	3	2	1	0
(a) calculate_fee function (4 marks)		Correct calculation of total fee using credit hours and fee per credit.	Correct logic but minor mistake in calculation or return value.	Function implemented but incorrect calculation.	Function exists but incomplete/incorrect logic.	Function missing or incorrect implementation.
(b) Dictionary Implementation (4 marks)		Dictionaries correctly store course fee and credit hours for all course choices (1,2,3).	Minor issue in dictionary values or structure.	Dictionaries implemented but incomplete or partially incorrect.	Basic dictionary structure present but incorrect usage.	Dictionary missing or incorrect implementation.
(c) Student Class (5 marks)	Class correctly implemented with all attributes (student_name, course_name, credit_hours) and proper structure.	Minor issue in attribute implementation.	Class implemented but missing one attribute or incorrect structure.	Basic class structure present but not functional.	Attempted class but incorrect implementation.	Class missing or incorrect.
(d) display_info Method (4 marks)		Method correctly displays all details, total fee and fee category with correct logic.	Minor issue in output or formatting.	Method implemented but missing part of logic (e.g., category).	Method present but incorrect logic.	Method missing or incorrect.
(e) Main Program & Module Usage (4 marks)		Correctly accepts input, retrieves data from dictionaries, calls module function, creates object, and stores multiple records in list.	Minor issue in flow or data handling.	Program works but missing one requirement (e.g., list or object).	Basic structure present but incorrect logic.	Program missing or incorrect.
(f) Full Implementation & Output (4 marks)		Correct execution, integrates module, applies logic, and displays complete and accurate output.	Works but minor formatting/output issues.	Works but missing part of logic or output.	Runs but incorrect output.	Totally wrong output or not functioning.

QUESTION 2 [25 MARKS]

a. Create a Streamlit interface for the **Student Marks Analysis System** with the following widgets:

- Text input for **Student Name**
- Radio button for **Gender** (Male / Female)
- Number input for **Marks** (range 0–100)
- A button labelled "**Submit**"

```
7   st.title("Student Marks Analysis System")
8   st.write("Course: Python Programming")
9
10  names = []
11  genders = []
12  marks = []
13
14  for i in range(3):
15      st.subheader("Student " + str(i + 1))
16
17      name = st.text_input("Student Name", key="name" + str(i))
18
19      gender = st.radio(
20          "Gender", ["Male", "Female"], key="gender" + str(i)
21      )
22
23      mark = st.number_input(
24          "Marks", min_value=0, max_value=100,
25          value=0, step=1, key="marks" + str(i)
26      )
27
28      names.append(name.strip())
29      genders.append(gender)
30      marks.append(mark)
31
32  if st.button("Submit"):
```

[4 marks]

b. Apply exception handling using try-except. Display an appropriate error message if:

- Student name is empty

```
33      try:
34          if "" in names:
35              raise ValueError("Student name cannot be empty.")
36
```

[4 marks]

c. Store the marks of **THREE (3)** students in a NumPy array and display:

- Means marks (2 decimal places)
- Maximum marks

```
37          marks_array = np.array(marks)
38          mean_marks = np.mean(marks_array)
39          maximum_marks = np.max(marks_array)
```

```
63          st.subheader("Analysis")
64          st.write(f"Mean Marks: {mean_marks:.2f}")
65          st.write(f"Maximum Marks: {maximum_marks}")
```

[4 marks]

d. Create a Pandas DataFrame to store:

- Students name
- Gender
- Marks
- Result(pass/fail)

```
51          df = pd.DataFrame({
52              "Name": names,
53              "Gender": genders,
54              "Course": ["Python Programming"] * 3,
55              "Marks": marks_array,
56              "Result": results
57          })
58
59          st.subheader("Student Data")
60          st.dataframe(df)
```

[4 marks]

e. Apply data processing by:

- Sorting marks in ascending order
- Displaying:
  - Pass if marks ≥ 50

```
68        sorted_df = df.sort_values("Marks", ascending=True)
69
70        st.subheader("Sorted Student Records")
71        st.dataframe(sorted_df)
```

```
42        results = []
43
44        for mark in marks:
45            if mark >= 50:
46                results.append("Pass")
47            else:
48                results.append("Fail")
```

[4 marks]

f. Show the **complete code** for the Student Marks Analysis System.

question2.py > ...

```
1   import streamlit as st
2   import numpy as np
3   import pandas as pd
4   import matplotlib.pyplot as plt
5
6   st.title("Student Marks Analysis System")
7   st.write("Course: Python Programming")
8
9   names = []
10  genders = []
11  marks = []
12
13  for i in range(3):
14      st.subheader("Student " + str(i + 1))
15
16      name = st.text_input("Student Name", key="name" + str(i))
17
18      gender = st.radio(
19          "Gender", ["Male", "Female"], key="gender" + str(i)
20      )
21
22      mark = st.number_input(
23          "Marks", min_value=0, max_value=100,
24          value=0, step=1, key="marks" + str(i)
25      )
26
27      names.append(name.strip())
28      genders.append(gender)
29      marks.append(mark)
30
31  if st.button("Submit"):
32      try:
33          if "" in names:
34              raise ValueError("Student name cannot be empty.")
35
36          marks_array = np.array(marks)
37          mean_marks = np.mean(marks_array)
38          maximum_marks = np.max(marks_array)
```

```
        results = []

        for mark in marks:
            if mark >= 50:
                results.append("Pass")
            else:
                results.append("Fail")

        df = pd.DataFrame({
            "Name": names,
            "Gender": genders,
            "Course": ["Python Programming"] * 3,
            "Marks": marks_array,
            "Result": results
        })

        st.subheader("Student Data")
        st.dataframe(df)

        st.subheader("Analysis")
        st.write(f"Mean Marks: {mean_marks:.2f}")
        st.write(f"Maximum Marks: {maximum_marks}")

        sorted_df = df.sort_values("Marks", ascending=True)

        st.subheader("Sorted Student Records")
        st.dataframe(sorted_df)

        st.subheader("Marks Graph")

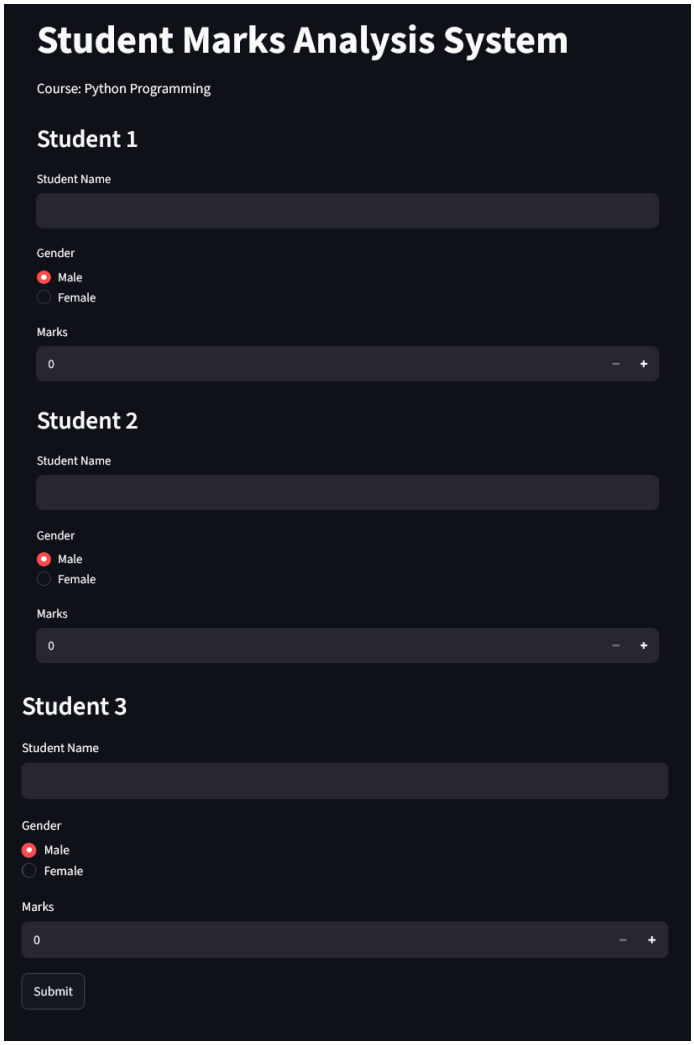
        plt.figure()
        plt.bar(range(3), marks_array, color="royalblue")
        plt.xticks(range(3), names)
        plt.xlabel("Student")
        plt.ylabel("Marks")
        plt.title("Student Marks")
        plt.ylim(0, 100)

        st.pyplot(plt.gcf())
        plt.close()

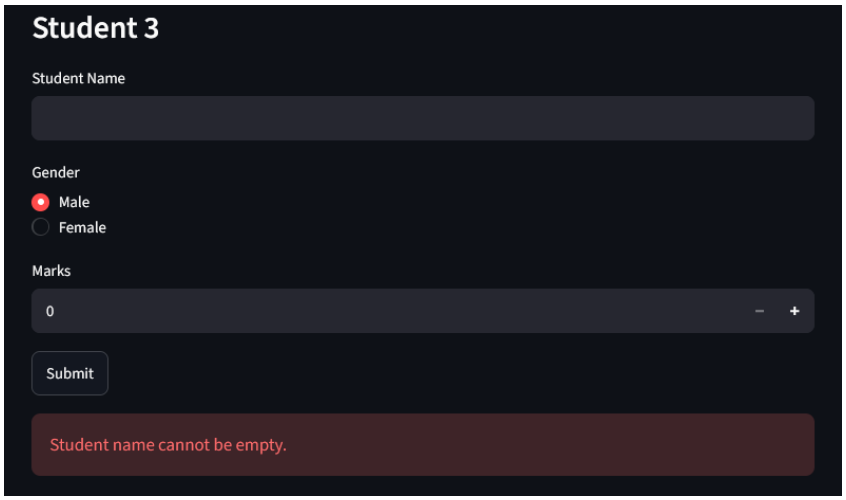
    except ValueError as error:
        st.error(str(error))
```

[5 marks]

Output :



Validation Error :



After Submiting Student Data :

Student Data

	Name	Gender	Course	Marks	Result
0	Sarveysh	Male	Python Programming	97	Pass
1	Tannus	Male	Python Programming	89	Pass
2	Varshini	Female	Python Programming	98	Pass

Analysis

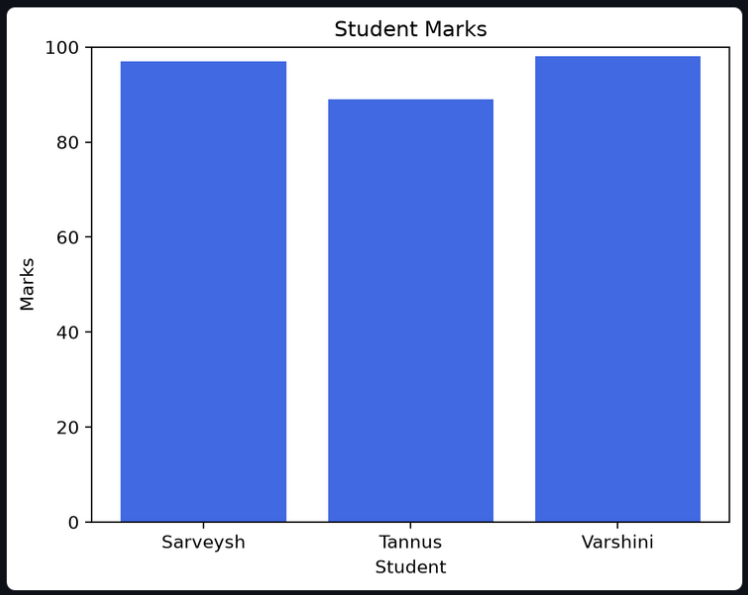
Mean Marks: 94.67

Maximum Marks: 98

Sorted Student Records

	Name	Gender	Course	Marks	Result
1	Tannus	Male	Python Programming	89	Pass
0	Sarveysh	Male	Python Programming	97	Pass
2	Varshini	Female	Python Programming	98	Pass

Marks Graph



Rubric for Students Marks Analysis System (Total: 25 Marks)

Component	5	4	3	2	1	0
(a) Streamlit Interface (4 marks)		Correct interface with all widgets (text input, radio, number input, button) and supports THREE students.	Minor issue in widget usage or layout.	Interface created but missing one component.	Basic interface present but incorrect usage.	Interface missing or incorrect.
(b) Exception Handling (4 marks)		Correct use of try-except to validate input and display appropriate error message.	Minor issue in validation or message.	Exception handling implemented but incomplete.	Basic try-except present but incorrect logic.	Exception handling missing or incorrect.
(c) NumPy Analysis (4 marks)		Correct use of NumPy array with accurate mean (2 decimal) and maximum calculation.	Minor issue in calculation or formatting.	NumPy used but incorrect calculation.	Basic NumPy structure present but incorrect usage.	NumPy not used or incorrect.
(d) Pandas DataFrame (4 marks)		DataFrame correctly created with all columns (Name, Gender, Course, Marks, Result).	Minor issue in structure or data.	DataFrame created but missing one column.	Basic DataFrame present but incorrect usage.	DataFrame missing or incorrect.
(e) Data Processing & Visualization (4 marks)		Correct sorting, result classification (Pass/Fail), and graph displayed properly.	Minor issue in sorting or graph display.	Processing done but missing one component (sorting/result/graph).	Basic processing present but incorrect logic.	Processing or graph missing/incorrect.
(f) Full Implementation & Output (5 marks)	Complete and correct program integrating Streamlit, exception handling, NumPy, Pandas, and graph with accurate output.	Works with minor formatting/output issues.	Works but missing one major component.	Runs but incorrect logic/output.	Attempted but not functioning properly.	No implementation or totally incorrect.